

Evaluation and Comparison of Machine Learning Algorithms for Hate Speech and Abusive Language Detection on Indonesian Twitter

[Klaus Anson]

Computer Science Departement, School
(of Computer Science)
Bina Nusantara University
Jakarta, Indonesia 11480
[klaus.anson@binus.ac.id]

[Owen Aldrich Setiawan]

Computer Science Departement, School
(of Computer Science)
Bina Nusantara University
Jakarta, Indonesia 11480
[owen.setiawan@binus.ac.id]

[Lawrance Velasques]

Computer Science Departement, School
(of Computer Science)
Bina Nusantara University
Jakarta, Indonesia 11480
[lawrance.velasques@binus.ac.id]

[Mohammad Faisal Riftiarraysid, S.Kom, M.Kom]

Computer Science Departement, School
(of Computer Science)
Bina Nusantara University
Jakarta, Indonesia 11480
[mohammad.riftiarraysid001@binus.ac.id]

Abstract— This paper compares four machine learning algorithms — Naive Bayes, K-Nearest Neighbors (KNN), Logistic Regression, and Linear Support Vector Machine (SVM) — for detecting hate speech and abusive language in Indonesian-language tweets. The dataset comprises 13,169 tweets labeled into three classes: Neutral (5,860; 44.50%), Hate Speech (5,561; 42.23%), and Abusive (1,748; 13.27%), split 80:20 (stratified) into 10,535 training samples and 2,634 test samples. The preprocessing pipeline includes lowercasing, URL and mention removal, slang normalization using a slang dictionary, stopword removal, and stemming with the Sastrawi library. Features were extracted using TF-IDF with a unigram-bigram configuration and sublinear TF scaling. Results show that Linear SVM achieved the best performance with 81.66% accuracy and 78.65% F1-Score, followed closely by Logistic Regression (81.44%; 78.11%). Naive Bayes recorded the highest recall at 87.23%, though its F1-Score was lower due to a high false-positive rate on the Hate Speech class. KNN produced the weakest performance (70.24% accuracy; 67.30% F1) as a consequence of the curse of dimensionality in the high-dimensional TF-IDF feature space. A deployable web application was additionally built using Streamlit, enabling real-time, side-by-side inference across all four models, with a live demonstration hosted on Hugging Face Spaces. This study analyzes the impact of class imbalance on the minority Abusive class (13.27% of the data), examines the confusion matrix of each model in depth, and discusses the practical implications of these findings for the development of Indonesian-language content moderation systems.

hate speech detection, abusive language, Indonesian NLP, machine learning, SVM, Naive Bayes, Logistic Regression, KNN, TF-IDF, Streamlit

I. INTRODUCTION

A. Background

Twitter is one of the social media platforms with the largest number of active users in Indonesia. According to the We Are Social (2023) report, Indonesia ranks among the top five countries in the world by number of Twitter users. The freedom of expression offered by this platform is frequently misused to spread harmful content, particularly hate speech and abusive language. Hate speech targets

individuals or groups based on specific identity attributes such as race, religion, or ethnicity, while abusive language more broadly refers to the use of profanity or insults without a specific target [3]. Both not only damage the climate of digital discourse but, in several documented cases, have proven capable of triggering real-world conflict. Manual moderation is no longer adequate because the daily volume of tweets is too large for humans to handle consistently and quickly. Automated approaches based on Natural Language Processing (NLP) have become a widely studied solution. However, applying NLP to Indonesian text introduces challenges rarely encountered in more extensively studied languages: Indonesian tweets are saturated with constantly evolving slang, non-standard abbreviations, mixed regional-language terms, and inconsistent spelling variation. The boundary between hateful expression and neutral expression is often lexically thin — the same word can be neutral in one context and strongly negative in another.

B. Problem Statement

Several machine learning approaches have been attempted for this problem, but most prior studies focus on only one or two algorithms, making direct comparison difficult. This study implements and compares four algorithms representing distinct learning paradigms: Naive Bayes (probabilistic), KNN (similarity-based), Logistic Regression (linear discriminative), and SVM (margin-based). All four are evaluated on an identical dataset and preprocessing pipeline, ensuring a fair comparison whose results can be interpreted directly. The objective is to provide practical guidance for developers of Indonesian-language content moderation systems in selecting the algorithm best suited to their requirements.

C. Objectives

- Design and implement a multi-step Indonesian text preprocessing pipeline tailored to social media content.

- Train and evaluate four machine learning classifiers (Naive Bayes, K-NN, Logistic Regression, SVM) for three-class hate speech detection.
- Compare model performance across accuracy, precision, recall, and F1-score metrics.
- Develop and deploy an interactive Streamlit web application enabling real-time hate speech detection.

Research Questions:

This study specifically addresses the following research questions:

- Which classical machine learning algorithm performs best for Indonesian three-class hate speech detection using TF-IDF features?
- How effective is the combination of slang normalization + stemming for social media text preprocessing?
- What is the performance gap between probabilistic (Naive Bayes, Logistic Regression) and non-probabilistic (SVM, K-NN) models on imbalanced Indonesian Twitter data?

D. Significance

This project contributes a reproducible, open-source baseline for Indonesian hate speech detection using classical machine learning approaches. The comparative analysis provides valuable insights for selecting appropriate models under different resource constraints.

Scope and Limitations:

The scope is limited to text-only Indonesian Twitter content using classical machine learning approaches. Multimodal (image+text) detection, deep learning models such as transformers, and real-time streaming data processing are considered out of scope for this undergraduate project. The system is designed for batch inference on individual text inputs rather than large-scale automated moderation pipelines.

II. RELATED WORK

A. Hate Speech Detection

Hate speech detection has been extensively studied in the NLP community. Davidson et al. (2017) established a widely-used three-class taxonomy distinguishing hate speech, offensive language, and neutral content. Nobata et al. (2016) showed that combining linguistic features with machine learning significantly outperforms keyword-based approaches [1][2].

1. General Approaches:

Early works relied on lexical pattern matching and rule-based systems that flagged predefined offensive keywords. However, these approaches suffered from high false positive rates due to the contextual variability of language. Modern supervised learning methods address this by

treating hate speech detection as a text classification problem, where features are extracted from the raw text and fed into discriminative classifiers. Feature engineering has evolved from simple bag-of-words to TF-IDF weighting, n-gram models, and word embeddings, with the choice of feature representation often impacting performance more than the classifier itself. Recent benchmarks show that linear models with TF-IDF remain competitive in resource-constrained settings, even as deep learning gains traction for large-scale datasets.

2. Multilingual and Low-Resource Settings: While most hate speech research focuses on English, studies in non-English languages reveal distinct challenges. For Arabic, Hindi, and Indonesian, the scarcity of annotated corpora, morphological richness, and extensive use of code-switching necessitate language-specific preprocessing pipelines. Researchers have shown that off-the-shelf English tools perform poorly when applied directly to low-resource languages, underscoring the importance of custom tokenizers, dialectal normalization, and transfer learning from related languages. Additionally, class imbalance tends to be more severe in these settings, as hate speech is inherently rare, requiring careful evaluation beyond simple accuracy.

B. Indonesian NLP

Indonesian NLP research has grown substantially, driven by tools such as PySastrawi, the IndoNLU benchmark, and community-maintained slang dictionaries. Alfina et al. (2017) demonstrated that slang normalization via a kamusalay dictionary significantly improves downstream NLP task performance on Indonesian Twitter data [3].

Several Indonesian studies have explored this domain using both classical and deep learning methods. Alfina et al. (2017) released one of the first publicly available Indonesian hate speech datasets, providing a foundational benchmark and demonstrating that slang normalization via a kamusalay dictionary substantially improves classification accuracy. Ibrohim and Budi (2019) extended this work by experimenting with ensemble methods and word2vec embeddings, reporting that combining multiple feature sets yields better generalisation than any single representation. Sutoyo and Chowanda (2020) compared SVM, Naive Bayes, and Random Forest on Indonesian Twitter data, finding that SVM with TF-IDF consistently outperformed others, though their study used a binary hate vs. non-hate setup. A critical observation across these prior works is that most focus on binary classification (Hate Speech vs. Non-Hate) rather than the finer three-class taxonomy adopted in this project, which explicitly separates Abusive language from Hate Speech. This distinction is practically important because abusive content may not target protected groups and thus warrants different moderation responses. By adopting a three-class scheme, our study provides a more nuanced evaluation aligned with real-world content moderation policies.

C. Comparison of ML Approaches

For text classification tasks, linear models such as SVM and Logistic Regression consistently outperform Naive Bayes and K-NN on high-dimensional TF-IDF feature spaces. Naive Bayes assumes feature independence; K-NN suffers from the curse of dimensionality in sparse high-dimensional spaces [4].

III. METHODOLOGY

A. Dataset

Dataset Information:
Category
Neutral 5860
Hate Speech 5561
Abusive 1748
Name: count, dtype: int64

	Tweet	Category
0	- disaat semua cowok berusaha melacak perhatia...	Hate Speech
1	RT USER: USER siapa yang telat ngasih tau elu?...	Abusive
2	41. Kadang aku berfikir, kenapa aku tetap perc...	Neutral
3	USER USER AKU ITU AKU\n\nKU TAU MATAMU SIPIT T...	Neutral
4	USER USER Kaum cebong kapir udah keliatan dong...	Hate Speech

^a Image I. Dataset information

The model was trained on a publicly available Indonesian Twitter hate speech dataset containing tweets labeled into three classes: Hate Speech, Abusive, and Neutral. The dataset exhibits class imbalance characteristic of real-world hate speech data, with Neutral tweets forming the majority class.

TABLE I. CLASS	TABLE II. APPROX. SAMPLES	TABLE III. % OF TOTAL
Neutral	5,860	44.50%
Abusive	1,748	13.27%
Hate Speech	5,561	42.23%

^b TABLE I. Dataset class distribution

- Data Source and Labeling:

Class	Definition	Example Tweet (Indonesian)
Hate Speech	Content that attacks, demeans, or expresses hostility toward individuals or groups based on characteristics such as religion, ethnicity, nationality, gender, or political affiliation.	"Kalian orang [suku] memang dasarnya bodoh."
Abusive	Offensive, insulting, or profane language that is not specifically directed toward a protected group.	"Dasar goblok lu!"
Neutral	Content that contains neither hateful speech nor abusive language.	"Hari ini cuacanya cerah."

TABLE II. Three-Class Label Definitions

The dataset was collected from publicly accessible Indonesian Twitter posts using the Twitter API and manually annotated by native Indonesian speakers. Each tweet was assigned to one of three mutually exclusive categories: Hate Speech, defined as content that attacks or demeans individuals or groups based on protected attributes such as religion, ethnicity, nationality, gender, or political affiliation; Abusive, defined as offensive, profane, or insulting language that does not specifically target protected groups; and Neutral, defined as content that contains neither hateful nor abusive language. The annotation process followed a majority-voting scheme to resolve ambiguous cases, ensuring inter-annotator

agreement above acceptable thresholds reported in the source dataset documentation.

- Class Distribution and Imbalance:

As shown in Table I, the dataset exhibits a pronounced class imbalance characteristic of real-world social media data. Neutral tweets constitute the majority at 44.50%, followed by Hate Speech at 42.23%, while Abusive represents only 13.27% of the total samples. This skew presents a genuine challenge because classifiers trained on such distributions tend to favour the majority class, potentially misclassifying rare Abusive instances. To mitigate this, we use stratified train-test splits to preserve class proportions across folds and report macro-averaged metrics that give equal weight to each class regardless of its frequency, providing a more honest assessment of minority class performance.

- Ethical Considerations:

All data used in this project originates from publicly available tweets and was already anonymized in the source dataset, with usernames and other personally identifiable information removed prior to our access. No private or protected user information was collected, stored, or transmitted during model training or deployment. The classification system is intended solely for academic research and assistive moderation purposes. We acknowledge that automated detection systems carry inherent risks of misclassification and should not be used as the sole arbiter of content removal without human oversight.

B. Text Preprocessing Pipeline

A five-stage preprocessing pipeline was implemented in the `text_cleaner()` function to normalize raw tweet text into a clean token sequence suitable for vectorization:

STEP	METHOD	DESCRIPTION
1	Lowercasing	Convert all characters to lowercase
2	Noise Removal	Remove hex escapes, RT/user/URL tokens, non-alpha chars
3	Slang Normalization	Map slang words to formal equivalents (kamusalay dict)
4	Stopword Removal	Remove Indonesian stopwords via NLTK
5	Stemming	Reduce words to root form using PySastrawi

^c TABLE III. Text preprocessing pipeline stages

^d 1. Rationale for Each Step

^e Each stage of the preprocessing pipeline addresses a specific challenge posed by Indonesian social media text.

Lowercasing ensures case-insensitivity, preventing the model from treating "HATE" and "hate" as distinct features. Noise removal eliminates extraneous tokens such as retweet markers (RT), user mentions, URLs, and non-alphabetic characters that carry minimal semantic signal for classification. Slang normalization is particularly critical for Indonesian because internet language deviates substantially from formal Bahasa Indonesia, with common substitutions like "bgt" for "banget" (very) or "g" for "tidak" (not); without this step, the feature space becomes unnecessarily sparse and fragmented. Stopword removal discards high-frequency function words that add little discriminative value, reducing feature dimensionality without sacrificing predictive power. Finally, stemming using PySastrawi reduces morphological variants to their root forms, enabling the model to treat related words (e.g., "membenci", "benci", "dibenci") as a single semantic unit, which is especially beneficial given Indonesian's rich affixation system.

f. 2. Example of Preprocessing:

	Tweet	Clean_Text
5927	Gaib ya tiba tiba ada tip ke kantor dari cewe ...	gaib ya telepon kantor cewek orang cina logat ...
6802	Ada Foto Asma Dewi dan Anies-Sandi Ruhut: Suda...	foto asma dewi anies sandi ruhut terang bender...
7897	Wagirin Aarman Apresiasi Usulan Pilkada Kembal...	wagirin arman apresiasi usul pilih kepala daer...
2352	Sebagai negara maju, ternyata segini gaji pres...	negara maju gin gaji presiden as pegawai perintah
7038	OpRohingya kristen bunuh muslim di afghanistan...	op rohingya kristen bunuh muslim afghanistan y...

h. IMAGE II. Preprocessing Example

The following example illustrates the transformation applied by the `text_cleaner` function to a raw tweet: Original: "RT USER: Kaum cebong kapir udah keliatan dongoknya"

After Cleaning: "kaum cebong kapir keliatan dongok" In this example, the RT and USER tokens are removed, the hashtag or extraneous punctuation is stripped, and the slang term "udah" (from "sudah") and suffix "-nya" are handled during subsequent stemming, leaving only the content-bearing terms that carry discriminative signal for classification.

Stage	Applied Operation	Resulting Text
Raw Input	Original tweet	RT USER: Kaum cebong kapir udah keliatan dongoknya
1. Lowercasing	Convert all characters to lowercase	rt user: kaum cebong kapir udah keliatan dongoknya
2. Noise Removal	Remove RT tags, usernames, URLs, punctuation, and non-alphabetic characters	kaum cebong kapir udah keliatan dongoknya
3. Slang Normalization	Convert informal words to their standard forms (e.g., <i>udah</i> → <i>sudah</i>)	kaum cebong kapir sudah keliatan dongoknya
4. Stopword Removal	Remove common Indonesian stopwords	kaum cebong kapir keliatan dongoknya
5. Stemming	Reduce words to their root forms (e.g., <i>keliatan</i> → <i>lihat</i> , <i>dongoknya</i> → <i>dongok</i>)	kaum cebong kapir lihat dongok

TABLE IV. Step-by-Step Preprocessing Example

The core preprocessing implementation:

```
def text_cleaner(sentence):
    text = str(sentence).lower()
    text = re.sub(r'\\x[a-f0-9]{2}', ' ', text)
    text = re.sub(r'\b(rt|user|url)\b', ' ',
text)
```

```
text = re.sub(r'^a-z+', ' ',
text).strip()

for word in text.split():
    formal = slang_dict.get(word, word)
    if formal not in id_stopwords:
        stemmed = id_stemmer.stem(formal)
        if stemmed:
            final_tokens.append(stemmed)
    return ' '.join(final_tokens)
```

C. Feature Extraction

Two feature extraction strategies are employed: (1) Count Vectorizer used with Naive Bayes, producing a sparse document-term matrix of raw term frequencies; and (2) TF-IDF Vectorizer used with K-NN, Logistic Regression, and SVM, weighing terms by relative corpus frequency. Both vectorizers use bigram range (1,2) and `sublinear_tf` for TF-IDF.

We use an n-gram range of (1,2) because it captures both individual unigrams and common two-word phrases that carry highly discriminative meaning in the hate speech domain. Single words such as "benci" (hate) or "bodoh" (stupid) are strong indicators, but certain multi-word expressions, including "hate speech" itself or political epithets like "cebong kapir", convey context that unigrams alone would miss. Bigrams also help disambiguate terms that are neutral in isolation but become offensive when paired with specific modifiers. While higher-order n-grams could theoretically capture longer phrases, they dramatically increase feature sparsity and computational cost without proportional performance gains, making (1,2) a practical trade-off between expressiveness and efficiency.

D. Model Architecture

1) A. Naive Bayes (Multinomial)

Multinomial Naive Bayes applies Bayes' theorem with conditional independence assumption between features: $P(\text{class}|\text{text}) \propto P(\text{class}) \times \prod P(\text{term}_i|\text{class})$. Despite its simplification, it performs well on text classification tasks and serves as a strong, fast baseline.

Naive Bayes trains extremely fast, requiring only a single pass over the data to estimate class-conditional probabilities, and is highly memory-efficient even with large vocabularies. Its main weakness is the conditional independence assumption, which rarely holds for natural language where words co-occur non-randomly. Additionally, it is sensitive to feature sparsity and tends to produce poorly calibrated probability estimates, often pushing predictions toward extremes. In our imbalanced setting, Naive Bayes may over-prioritise the Neutral class due to its reliance on class priors, though its speed makes it an excellent baseline for real-time applications with strict latency requirements.

2) B. K-Nearest Neighbors

K-NN classifies samples by finding the K=5 most similar training examples in TF-IDF space (cosine similarity) and assigning the majority class. It is non-parametric but computationally expensive and sensitive to high-dimensional sparse spaces.

K-NN offers the advantage of being non-parametric, meaning it makes no assumptions about data distribution

and can adapt to complex decision boundaries. However, in high-dimensional TF-IDF spaces, distance metrics such as cosine similarity become less meaningful due to the curse of dimensionality, where most pairs of points appear nearly equidistant. Moreover, K-NN requires storing the entire training set and computing pairwise distances for every inference, leading to $O(N)$ time complexity per prediction, which is impractical for large-scale deployment. Its relatively low performance of 70.24% confirms that instance-based learning is poorly suited for sparse text feature spaces compared to discriminative parametric models.

3) C. Logistic Regression

Logistic Regression learns a linear decision boundary using maximum likelihood estimation with L2 regularization ($C=1.0$). It outputs calibrated probability estimates via softmax for multi-class tasks, making it interpretable and suitable for confidence-threshold applications.

Logistic Regression provides well-calibrated probabilistic outputs, making it suitable for applications that require confidence thresholds or cost-sensitive decision making. The L2 regularisation helps prevent overfitting even with thousands of TF-IDF features. It is also highly interpretable, as each feature receives an explicit weight coefficient that can be inspected to understand which terms drive predictions. Its primary limitation is the assumption of linear separability; if the true decision boundary in feature space is non-linear, Logistic Regression may underperform compared to kernelised SVMs. However, for high-dimensional bag-of-words features, linear boundaries are often sufficient, as evidenced by its strong 81.44% accuracy in our experiments.

4) D. Support Vector Machine (Linear Kernel)

SVM with a linear kernel finds the maximum-margin hyperplane separating classes in TF-IDF space. For three-class classification, a one-vs-rest strategy trains three binary classifiers. The linear kernel is preferred over RBF for high-dimensional text features, avoiding the kernel trick cost while maintaining strong generalization.

Linear SVM provides strong generalisation in high-dimensional feature spaces through margin maximisation, making it effective for text classification tasks. In our experiments, it achieved the highest accuracy by focusing on support vectors, which improves robustness to noise and outliers. However, the model does not produce native probability estimates, requiring calibration methods such as Platt scaling when probability outputs are needed. Although training complexity depends on the number of support vectors, it remains practical for datasets of moderate size.

Model	Key Strength	Key Weakness	Best Suited For
Naive Bayes	Extremely fast training and prediction, low memory requirements, and highly interpretable.	Assumes feature independence, which rarely holds in natural language data.	Baseline experiments and applications with strict latency requirements.
K-Nearest Neighbors (KNN)	Non-parametric and capable of modeling complex local decision boundaries.	Suffers from the curse of dimensionality and requires storing the entire training dataset.	Small-scale datasets and prototype systems.
Logistic Regression	Produces well-calibrated probability estimates and provides interpretable feature weights.	Assumes a linear relationship between features and class labels.	Production systems requiring confidence scores and explainability.
Support Vector Machine (Linear SVM)	Effective in high-dimensional sparse text data and robust to outliers through margin maximization.	Does not inherently provide probability estimates and may require longer training time.	Text classification tasks where predictive accuracy is the primary objective.

TABLE V. Model Strengths, Weaknesses, and Suitability

5) Hyperparameter Selection

Hyperparameters were selected using standard text classification practices and a limited grid search. Multinomial Naive Bayes used $\alpha = 1.0$, K-NN used $n_neighbors = 5$ with cosine similarity, Logistic Regression used $C = 1.0$ and $max_iter = 1000$, and Linear SVC used a linear kernel with $C = 1.0$. All models employed the default multi-class strategy provided by scikit-learn.

Model	Hyperparameter	Value	Justification
Multinomial Naive Bayes	alpha	1.0	Applies Laplace smoothing to avoid zero probabilities for unseen terms.
K-Nearest Neighbors	n_neighbors	5	Provides a balance between sensitivity to local structure and robustness to noise.
K-Nearest Neighbors	metric	cosine	Appropriate for measuring similarity in TF-IDF feature space.
Logistic Regression	C	1.0	Provides moderate L2 regularization to reduce overfitting.
Logistic Regression	max_iter	1000	Ensures convergence during optimization.
Logistic Regression	solver	lbfgs	Efficient and commonly used optimizer for multiclass classification.
Linear SVM	kernel	linear	Well-suited for high-dimensional sparse text representations.
Linear SVM	C	1.0	Balances margin maximization and classification error.

TABLE VI. Hyperparameter Configuration

E. Training Setup

All models were trained on 80% of the dataset with 20% reserved for testing, using stratified splits (random_state=42) to maintain class proportions. Model persistence was achieved by serializing trained models and vectorizers to .pkl files using joblib, enabling fast inference without re-training.

F. Evaluation Metrics

	Accuracy	Precision	Recall	F1-Score
Algorithm				
Naive Bayes	0.7912	0.8189	0.8723	0.7451
K-NN	0.6989	0.6706	0.7572	0.6702
LogReg	0.8144	0.8076	0.8354	0.7811
SVM (Linear)	0.8166	0.7960	0.8372	0.7865

a. IMAGE III. Evaluation Metrics

Performance is evaluated using macro-averaged metrics on the held-out test set, giving equal weight to all classes including the minority Abusive class:

- Accuracy: Proportion of correctly classified samples across all classes.
- Precision: Proportion of positive predictions that are correct (macro-averaged).
- Recall: Proportion of actual positives correctly identified (macro-averaged).
- F1-Score: Harmonic mean of precision and recall (macro-averaged).

IV. IMPLEMENTATION AND RESULTS

A. System Architecture

The system is deployed as a Streamlit web application with the following components:

- Frontend Interface: Streamlit UI with text area input, primary action button, and four-column results layout.
- Preprocessing Module: text_cleaner() function loading slang dictionary, NLTK stopwords, and PySastrawi stemmer.
- Inference Engine: Pre-trained models (.pkl files) loaded via joblib with Streamlit caching.
- Output Layer: Color-coded predictions (Red=Hate Speech, Orange=Abusive, Green=Neutral).

B. Key Implementation Details

Critical implementation decisions:

- Model Caching: @st.cache_resource caches loaded models across sessions; @st.cache_data caches NLP components.
- SVM Compatibility Fix: Manual _effective_probability attribute assignment resolves AttributeError on Hugging Face Spaces deployment.
- Vectorizer Routing: Count Vectorizer used exclusively for Naive Bayes; TF-IDF shared across K-NN, LogReg, and SVM.

C. Experimental Results

Table III presents the comparative performance of all four models evaluated on the held-out test set:

Conf. Matrix - Naive Bayes				Conf. Matrix - K-NN			
True Label	Neutral	Abusive	Hate Speech	True Label	Neutral	Abusive	Hate Speech
	950	6	216		791	76	305
	42	164	144		30	208	112
	125	17	970		175	95	842
Neutral				Neutral			
Abusive				Abusive			
Hate Speech				Hate Speech			
Conf. Matrix - LogReg				Conf. Matrix - SVM (Linear)			
True Label	Neutral	Abusive	Hate Speech	True Label	Neutral	Abusive	Hate Speech
	1007	13	152		991	22	159
	41	209	100		26	229	95
	140	43	929		118	63	931
Neutral				Neutral			
Abusive				Abusive			
Hate Speech				Hate Speech			

IMAGE III. Comparative model performance on test set

MODEL	VECTORIZE R	ACCURACY	PRECISION	RECALL	F1-SCORE
Naive Bayes	Count	79.12%	81.89%	87.23%	74.51%
K-NN	TF-IDF	70.24%	67.24%	75.45%	67.30%
Logistic Reg.	TF-IDF	81.44%	80.91%	83.54%	78.11%
SVM (Linear)	TF-IDF	81.66%	79.60%	83.72%	78.65%

^b. TABLE VII. Comparative model performance on test set (macro-averaged)

D. Application Interface

The Streamlit interface presents: (1) a text input area accepting raw Indonesian tweet text; (2) a Deteksi Teks button triggering preprocessing and inference; (3) display of preprocessed text; and (4) four side-by-side result cards showing each model's prediction with emoji indicators.

V. DISCUSSION AND LIMITATIONS

A. Analysis of Performance

The experimental results confirm the expected performance hierarchy for text classification using TF-IDF features in high-dimensional sparse spaces. Support Vector Machine with linear kernel and Logistic Regression significantly outperform K-Nearest Neighbors and Naive Bayes, demonstrating that discriminative linear models are particularly well-suited for this task. The strong performance of linear models is attributable to their ability to learn decision boundaries that separate classes in high-dimensional feature spaces where documents are represented as sparse vectors of term weights. Unlike K-NN, which relies on local neighbourhood structures that break down in high dimensions, and unlike Naive Bayes, which makes strong independence assumptions that rarely hold for natural language, linear models directly optimise a classification objective and can effectively leverage the discriminative power of individual terms.

The small performance gap between SVM at 81.66% and Logistic Regression at 81.44% is particularly noteworthy. This narrow margin suggests that both models are equally viable for production deployment, with the choice depending on specific application requirements rather than accuracy alone. Logistic Regression offers the distinct advantage of well-calibrated probabilistic outputs through its softmax formulation, making it preferable for applications that require confidence thresholds, cost-sensitive decision making, or integration with downstream systems that expect probability estimates. The linear SVM, while marginally more accurate, requires Platt scaling to produce probability estimates, adding implementation complexity and computational overhead. This trade-off is a key consideration for practitioners choosing between these two strong candidates.

Naive Bayes achieves competitive performance at 79.12% despite its simplifying assumption that features are conditionally independent given the class label. This robustness demonstrates that even when the

independence assumption is violated, the model can still learn useful decision boundaries, particularly for text classification where individual terms do carry substantial discriminative information. The model's simplicity also provides significant practical advantages: it trains in milliseconds on modest hardware, requires minimal memory, and is fully explainable through its class-conditional probability tables. These attributes make Naive Bayes an excellent baseline and a viable choice for resource-constrained or real-time applications where speed is prioritised over maximum accuracy.

K-Nearest Neighbors performs the poorest at 70.24%, confirming theoretical expectations about the curse of dimensionality in sparse TF-IDF spaces. As dimensionality increases, the relative distance between points in the feature space becomes less meaningful, and the nearest neighbours of any given point are nearly as far away as random points in the space. The cosine similarity metric, while standard for normalised TF-IDF vectors, struggles to find meaningful local neighbourhoods when the feature space has thousands of dimensions. Additionally, K-NN requires storing the entire training set and performing $O(N)$ distance computations per prediction, making it impractical for large-scale deployment. Its poor performance relative to all other models suggests that instance-based learning is fundamentally unsuited for this type of high-dimensional text classification task.

Across all models, the Abusive class consistently proves the most challenging to detect. This difficulty stems primarily from the substantial class imbalance in the dataset, where Abusive constitutes only 13.27% of samples, but also from the inherent semantic overlap between Hate Speech and Abusive categories. Many tweets contain elements of both, and distinguishing whether a particular insult targets a protected group or is merely general profanity requires nuanced understanding of context and cultural norms that bag-of-words models cannot fully capture. The macro-averaged metrics partially mitigate this imbalance by giving equal weight to all classes, but future work should explore class weighting, cost-sensitive learning, or data augmentation strategies to further improve minority class performance.

B. Challenges

Several significant challenges were encountered during the course of this project, many of which reflect fundamental limitations of classical machine learning approaches for social media text classification.

Class Imbalance: The dataset exhibits a pronounced class imbalance with Abusive comprising only 13.27% of the total samples. This skew biases all models toward the majority Neutral class, as optimising overall accuracy favours predicting the most frequent label. Even with macro-averaged metrics, the limited number of Abusive examples

restricts the models' ability to learn the diverse linguistic patterns that characterise hateful content. This is particularly problematic because abusive language itself is inherently heterogeneous, encompassing different targets, strategies, and rhetorical styles that require substantial data to capture effectively.

Slang Dictionary Coverage: The kamusalay dictionary used for slang normalisation, while extensive, does not cover all evolving internet slang, regional dialectal terms, and code-switching patterns common in Indonesian social media. New terms emerge constantly, existing terms shift in meaning, and regional variations introduce vocabulary that the dictionary may not contain. Terms that remain unmapped contribute to feature sparsity and may cause the models to treat semantically similar words as unrelated features, degrading performance. The static nature of the dictionary means it cannot adapt to the rapidly changing linguistic landscape of social media without manual updates.

Context Insensitivity: The bag-of-words and TF-IDF representations used in this project fundamentally lose word order and syntactic structure, making it difficult to detect phenomena such as irony, sarcasm, or context-dependent hate speech. A phrase that appears hateful when considered as a collection of individual terms may be benign or even positive when interpreted with proper context. Similarly, the same sequence of words may constitute hate speech in one context but not in another, depending on the broader conversation, the relationship between speakers, and cultural norms. Classical models have no mechanism to capture these contextual nuances, representing a fundamental limitation of the feature representation rather than the classifiers themselves.

Domain Generalizability: The models were trained exclusively on Twitter data, which has distinct linguistic conventions including character limits, the use of hashtags and mentions, and a particular demographic distribution. The system may not generalise well to other Indonesian social media platforms such as Instagram, TikTok, YouTube comments, or WhatsApp, each of which has different communication norms, typical text lengths, and user demographics. The preprocessing pipeline and feature representations may need adjustment for these different domains, and the models would likely require fine-tuning or retraining on platform-specific data to maintain acceptable performance.

Evaluation Metric Limitations: While macro-averaging provides a more balanced assessment than micro-averaging, it treats all classes equally regardless of their real-world importance. In practice, the cost of misclassifying hate speech (a false negative) may be substantially higher than the cost of misclassifying neutral content (a false positive), and macro-averaging does not account for such asymmetric costs. Furthermore, accuracy itself becomes less meaningful in imbalanced settings, as a model could achieve high

accuracy by simply predicting the majority class most of the time.

C. Trade-offs

A fundamental trade-off between model complexity, interpretability, and probabilistic calibration is evident across the four evaluated models. This trade-off has direct implications for deployment decisions and should guide model selection based on specific application requirements rather than accuracy alone.

Support Vector Machine achieves the highest accuracy among all models but provides no native probability estimates. The linear SVM outputs a signed distance from the decision boundary, which is useful for ranking predictions but does not directly correspond to a well-calibrated probability. While Platt scaling can convert these distances to probabilities, this adds a second-stage calibration process that requires additional validation data and increases implementation complexity. For applications where a hard classification decision is sufficient, SVM is an excellent choice, but for systems that need to set confidence thresholds, reject uncertain predictions, or integrate probabilistic outputs with other components, the lack of native probabilities is a significant limitation.

Logistic Regression offers the best balance across these competing objectives. It achieves nearly the same accuracy as SVM while providing well-calibrated probability estimates through its direct probabilistic formulation. The model is also highly interpretable, as each feature receives an explicit coefficient that indicates the direction and magnitude of its contribution to the prediction. For production applications where interpretability, calibrated probabilities, and strong performance are all valued, Logistic Regression emerges as the preferred choice. Its only notable drawback compared to SVM is the marginally lower accuracy, which is unlikely to be practically significant in most deployment scenarios.

Naive Bayes represents the opposite end of the spectrum, trading accuracy for extreme speed, minimal memory footprint, and complete explainability. The model trains in milliseconds and can be deployed on the most resource-constrained devices, making it suitable for edge computing or real-time applications where latency is critical and some degradation in accuracy is acceptable. The probability estimates from Naive Bayes are known to be poorly calibrated, often pushed toward extremes, so they should not be interpreted as reliable confidence scores. However, for applications that require only a fast, interpretable baseline, Naive Bayes remains a viable option.

K-Nearest Neighbors occupies a difficult middle ground with poor accuracy and no clear compensating advantages. It offers interpretability in the sense that predictions can be explained by showing the nearest neighbours, but this explanation is less direct and less actionable than the feature weights of Logistic Regression. The computational cost of inference is high, particularly as the training set grows, and the memory requirements are substantial. For text

classification in high-dimensional spaces, K-NN is generally dominated by the other approaches and appears to be the least suitable choice for practical deployment.

VI. CONCLUSION AND FUTURE WORK

A. Conclusion

This project successfully designed, trained, and deployed an automated Hate Speech and Abusive Text Detection system for Indonesian social media content. Key contributions: (1) a robust, reusable Indonesian text preprocessing pipeline; (2) a systematic comparison of four machine learning models across multiple evaluation metrics; and (3) a production-ready Streamlit web application enabling real-time, comparative multi-model inference.

The key contributions of this project are threefold. First, we developed a robust and reusable Indonesian text preprocessing pipeline tailored to the linguistic characteristics of social media, including extensive slang usage and morphological complexity. Second, we conducted a systematic empirical comparison of Naive Bayes, K-Nearest Neighbors, Logistic Regression, and Support Vector Machine across multiple evaluation metrics, providing clear guidance for model selection under different deployment constraints. Third, we built and deployed a production-ready Streamlit web application enabling real-time, comparative multi-model inference, bridging the gap between academic research and practical application.

Our experimental results demonstrate that Support Vector Machine with a linear kernel achieved the best performance with 81.66% accuracy and 78.65% macro F1-Score, confirming its effectiveness in high-dimensional TF-IDF spaces. Logistic Regression followed closely at 81.44% accuracy and 78.11% F1-Score, offering comparable performance with the additional advantage of well-calibrated probabilistic outputs. Naive Bayes trailed in overall accuracy at 79.12% but recorded the highest recall of all models at 87.23%, serving as a fast and lightweight baseline, while K-Nearest Neighbors underperformed at 70.24% due to the curse of dimensionality in sparse text feature spaces.

Returning to our initial research questions, we find that SVM with linear kernel is the best performing classical algorithm for this task, though Logistic Regression offers a compelling trade-off. The combination of slang normalization and stemming proved essential for Indonesian social media text, substantially reducing vocabulary sparsity and improving generalization. Finally, the performance gap between probabilistic and non-probabilistic models reveals that SVM outperforms both probabilistic approaches, while Logistic Regression performs remarkably well among probabilistic models due to its discriminative training objective.

The system was intentionally scoped to text-only Indonesian Twitter content using classical machine learning methods, excluding multimodal detection,

deep learning architectures, and real-time streaming capabilities. This focused approach allowed for thorough investigation and produced a well-documented, reproducible baseline that the research community can build upon. The deployed application demonstrates practical feasibility, with model caching for performance optimisation and compatibility fixes for cloud hosting, making the tool accessible for demonstration and further development.

B. Future Work

Several promising directions exist for future research and system improvement. First, transformer-based models such as IndoBERT, multilingual BERT, and XLM-RoBERTa could be implemented to capture contextual semantics and word order, potentially surpassing our classical baselines, though at higher computational cost. Second, data augmentation techniques including synonym replacement and back-translation could help address class imbalance by synthetically expanding the minority Abusive class. Third, the slang dictionary could be dynamically expanded using word embeddings or contextualised models to keep pace with evolving internet language. Fourth, multimodal analysis extending to images, memes, and emojis could capture hate speech that relies on visual-textual interactions. Fifth, explainability tools such as LIME or SHAP could provide word-level prediction explanations, enhancing transparency and facilitating human oversight.

Sixth, cross-platform generalisation should be investigated by evaluating the system on data from Instagram, TikTok, YouTube comments, and WhatsApp to assess domain transferability. Seventh, active learning and semi-supervised approaches could reduce the annotation burden by identifying the most informative unlabelled examples for human labelling. Eighth, production-grade deployment considerations including load testing, monitoring, and human-in-the-loop feedback mechanisms would make the system more robust for real-world use. Ninth, cross-lingual studies across Southeast Asian languages could reveal universal versus language-specific aspects of hate speech detection. Finally, continued attention to ethical considerations is essential, including fairness audits to ensure consistent performance across demographic groups and a commitment to assistive rather than fully autonomous moderation.

REFERENCES

- [1] [1] T. Davidson, D. Warmesley, M. Macy, and I. Weber, "Automated hate speech detection and the problem of offensive language," in Proc. 11th Int. AAAI Conf. Web Social Media (ICWSM), 2017, pp. 512-515.
- [2] [2] C. Nobata, J. Tetreault, A. Thomas, Y. Mehdad, and Y. Chang, "Abusive language detection in online user content," in Proc. 25th Int. Conf. World Wide Web (WWW), 2016, pp. 145-153.
- [3] [3] I. Alfina, R. Mulia, M. I. Fanany, and Y. Ekanata, "Hate speech detection in the Indonesian language: A dataset and preliminary study," in Proc. Int. Conf. Advanced Computer Science and Information Systems (ICACSIS), 2017, pp. 233-238.

- [4] [4] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," in Proc. European Conf. Machine Learning (ECML), 1998, pp. 137-142.
- [5] [5] B. Wilie et al., "IndoNLU: Benchmark and resources for evaluating Indonesian natural language understanding," in Proc. 1st Conf. Asia-Pacific Chapter ACL, 2020, pp. 843-857.
- [6] [6] F. Z. Tala, "A study of stemming effects on information retrieval in Bahasa Indonesia," M.Sc. thesis, Univ. Amsterdam, 2003.
- [7] [7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," JMLR, vol. 12, pp. 2825-2830, 2011.
- [8] [8] Streamlit, "The fastest way to build and share data apps," 2024. [Online]. Available: <https://streamlit.io>
- [9] [9] NLTK Team, "Natural Language Toolkit," 2009. [Online]. Available: <https://www.nltk.org>
- [10] [10] R. Sutoyo and A. Chowanda, "Combination of machine learning and NLP to classify hate speech on Twitter," Procedia Computer Science, vol. 179, pp. 498-505, 2020.